

CampusConnect

INTRODUCTION:

CampusConnect is a versatile and user-friendly web application designed to streamline college community engagement and academic management. Suitable for students, faculty, and administrators, it offers a wide range of interactive features—including event registrations, club directories, academic forums, and peer-to-peer announcements. Users can fully tailor their profiles to match their academic interests, while robust security features like encrypted authentication help protect data integrity. CampusConnect also integrates seamlessly with campus databases to automate workflows and notifications. With built-in dashboard tools, administrators can generate engagement reports and manage campus activities for deeper analysis. Its flexibility, ease of use, and powerful features make it an ideal choice for college communities seeking an efficient platform to stay connected.

SCENARIO Based Case Study:

Meet Rahul, a first-year undergraduate student pursuing a Bachelor of Technology in Computer Science and Business Systems at Knowledge Institute of Technology in Salem. With a demanding academic schedule, he values efficiency and organization in navigating campus life. Rahul is responsible for keeping up with his coursework while also wanting to participate in college volunteering activities, join study groups, and plan trips with his close-knit group of friends, which requires him to stay updated on campus events and peer networks.

Rahul discovers CampusConnect, a comprehensive web application designed to streamline college community engagement. Intrigued by its features, Rahul decides to explore how CampusConnect can help him enhance his campus experience.

- **User Registration and Authentication:** Rahul registers an account on CampusConnect, providing his academic details and club preferences. He logs in securely using his credentials, ensuring that his data remains private and secure.
- **Event and Club Directory:** Rahul explores CampusConnect's extensive directory, which includes detailed information about campus organizations, upcoming hackathons, and volunteer drives. He can easily access this information to join events and answer his queries effectively.
- **Community Forums:** CampusConnect offers a discussion system that allows Rahul to manage his interactions with peers efficiently. He can track study group communications, schedule project meetings, and maintain a detailed student profile to personalize his networking.
- **Engagement Tracking:** CampusConnect provides Rahul with tools to track his extracurricular activities and participation metrics. He can analyze his active memberships and identify opportunities for growth, helping him make informed decisions about his free time.
- **Peer Networking:** Rahul can connect with his friends directly through CampusConnect, streamlining the process of organizing group activities and ensuring timely coordination.

for study sessions or weekend trips to places like Kodaikanal.

- **Rahul's Experience:** Thanks to CampusConnect, Rahul can now manage his college activities more efficiently and effectively. He can focus on building strong relationships with his peers, participating in campus life, and driving his personal growth. CampusConnect has become an indispensable tool in Rahul's daily routine, helping him achieve his academic and social goals with ease and confidence.

SYSTEM REQUIREMENTS:

Software Requirements:

To ensure smooth development, deployment, and usage of the Web-Application, certain system prerequisites must be met. These requirements are categorized into software, setup, and hardware specifications, all of which contribute to building a robust and scalable platform.

1. Software Requirements

These are the essential tools and platforms required to develop, test, and run the application efficiently.

- Operating System: Windows 10/11 macOS, or Linux Supports cross-platform development and testing.
- Node.js (v16 or above): Provides the runtime environment for building and managing frontend logic and Powers the server-side logic and handles API routing.
- npm (v8 or above): A package manager required to install dependencies for React and related libraries.
- React.js: A JavaScript library for building dynamic and responsive user interfaces.
- Browser: Google Chrome / Firefox (latest version) For rendering and testing the UI in real-time.
- Express.js: A lightweight web framework for building RESTful APIs.
- MongoDB: A NoSQL database used to store structured and unstructured data related to users, events, and campus resources.
- Postman: Tool for testing APIs during development.
- Visual Studio Code: Preferred code editor with built-in Git and terminal support.

2. Hardware Requirements

Describes the minimum and recommended specifications needed to support the development and usage of the application.

- Processor: Intel Core i5 (8th Gen or above) / AMD Ryzen 5 or better Ensures fast compilation and multitasking during development.
- RAM: Minimum 8 GB (16 GB recommended) - For handling development servers, IDEs, and browser testing simultaneously.
- Storage: At least 1 GB free space Required for package installations, MongoDB setup, and local project files.
- Display: 1366x768 or higher Recommended for optimal coding experience and application layout visualization.

PROJECT ARCHITECTURE:

In this architecture, the web application is divided into core microservices. The Frontend Web App communicates through an API Gateway Service. The API Gateway routes requests to specific Backend Services, such as the Authentication Service, Event Service, and User Service. These backend services then communicate with their respective databases (User Database, Event Database, Forum Database) and any necessary Third-party Services (such as Notification Services or Cloud Storage).

TECHNICAL ARCHITECTURE:

In this architecture, the flow begins with a user interacting with the CampusConnect frontend. The user submits a form or requests data through the frontend interface.

The CampusConnect frontend then sends a request to the CampusConnect backend. The backend processes the request and the associated data.

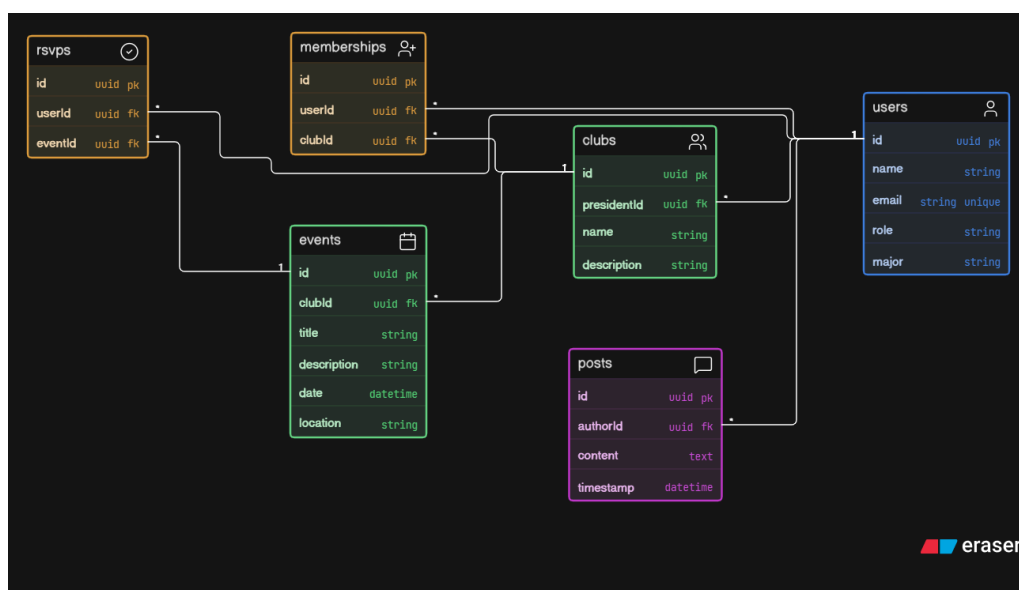
The backend communicates with a database to store the submissions securely. The database stores the submitted data, including the profiles, posts, and event RSVPs provided by the user.

Additionally, the backend may utilize a file storage system to store any files or attachments associated with the submissions. This can include uploaded profile images, club documents, or any other files relevant to the platform.

Once the backend has processed the request, it sends a response back to the CampusConnect frontend. The frontend then displays the response to the user.

ER DIAGRAM:

The Entity-Relationship (ER) diagram for CampusConnect represents the various entities and their relationships within the college engagement system. Here is a description of the key components of the ER diagram:



1. Entities:

- User: Represents students and faculty in the system, with attributes such as user ID, name, email, role, and major.
- Event: Represents campus events, with attributes such as event ID, title, description, date, and location.
- Club: Represents student organizations, with attributes such as club ID, name, description, and president ID.
- Post: Represents forum discussions, with attributes such as post ID, content, timestamp, and author ID.

2. Relationships:

- User RSVP: Represents the relationship between users and events, indicating which events a user is attending.
- User Membership: Represents the relationship between users and clubs, indicating which clubs a user has joined.
- User Post: Represents the relationship between users and posts, indicating which user authored a specific forum post.
- Club Event: Represents the relationship between clubs and events, indicating which club is hosting an event.

3. Attributes:

- Each entity has its own set of attributes that describe its properties.
- For example, the Event entity has attributes such as event ID, title, description, and date.

4. Primary Keys:

- Each entity has a primary key that uniquely identifies each record in the entity.
- For example, the User entity's primary key is the user ID.

5. Foreign Keys:

- Foreign keys are used to establish relationships between entities.
- For example, the Post entity may have a foreign key that references the user ID to link posts to their authors.

Key Features:

- **User Registration and Profiles:** Allow users to create accounts, set up profiles, and manage their academic information.
- **Event Management:** Allow users to search, browse, and RSVP to campus events based on their interests, including filtering by date and organizing club.
- **Discussion Forums:** Facilitate secure online discussions, supporting peer-to-peer academic queries and announcements.
- **Club Directory:** Enable users to discover and join student organizations, and allow club admins to manage member lists.
- **Activity History:** Maintain a record of past and upcoming events for users to reference and manage.

ROLES AND RESPONSIBILITIES:

User (Student/Faculty):

- **Registration and Account Management:** Users are responsible for creating an account, providing accurate personal information, and managing their account details, including profile updates and password security.
- **Event Search and RSVP:** Users can search for available events based on their preferences, such as location, date, time, and type of activity. They are responsible for selecting the desired event, booking it, and providing any necessary information.
- **Communication:** Users may need to communicate with club heads or the app's support team for inquiries, changes, or cancellations related to their events. They should utilize the provided communication channels appropriately and adhere to any specified guidelines.
- **Community Contributions:** Users have the responsibility to provide constructive input in academic forums and adhere to community guidelines when posting.

Admin (College Admin/Club Head):

- **System Management:** The admin is responsible for overall management and administration of the CampusConnect app. This includes monitoring system performance, ensuring data security and privacy, and managing app settings and configurations.
- **User Management:** The admin is responsible for managing user accounts, including registration, verification, and account moderation. They may handle tasks such as account approvals, suspensions, or terminations when necessary.
- **Event and Club Management:** The admin oversees the registration, verification, and approval process for clubs and events listed on the platform. They ensure the accuracy and quality of the activities provided to the student body.

User Flow:

Customer (Student) Flow

- Registration & Account - Create and manage account details securely.
- Search & RSVP - Find events by filters and complete registrations.
- Communication - Contact club heads/support for queries or changes.
- Community - Participate in forums to help peers and discuss academics.

Admin Flow

- System Management - Oversee app performance, settings, and security.
- User Management - Verify, approve, suspend, or manage user accounts.
- Event Management - Verify and maintain quality of campus events and club pages.

MVC PATTERN:

The CampusConnect application follows the Model-View-Controller (MVC) architectural pattern, a software design approach that separates an application into three interconnected layers. This separation allows for modularity, easier maintenance, and scalability.

- **Model Layer (Data Layer):** The Model layer is responsible for handling all data-related logic. This includes the definition of data schemas and the operations performed on the database using those schemas. The models are implemented using Mongoose, which

provides a schema-based solution to model application data for MongoDB.

- **Controller Layer:** The Controller layer acts as an intermediary between the view (routes) and the model. It receives incoming requests, processes the input (which may include validation or transformation), calls the appropriate methods from the model, and then returns a response to the client.
- **View Layer (Routing Layer):** In the context of a backend REST API, the View is implemented as the routing layer, where various endpoints are defined. These endpoints determine how the backend responds to different HTTP requests (GET, POST, PUT, DELETE) and are responsible for invoking the appropriate controller functions.

Advantages of Using MVC in This Project

- Separation of Concerns: Each layer has a clearly defined responsibility, improving readability and maintainability.
- Scalability: New features can be added easily by creating new routes, controllers, and models.
- Reusability: Logic in controllers and models can be reused across multiple parts of the application.
- Testing: Each layer can be tested independently, especially the controllers and models.
- Collaboration-Friendly: Multiple developers can work simultaneously on different layers without conflict.

Project Setup And Configuration.

- Creating project folder
 1. Create a new folder with your <project name>.
 2. Inside that folder create two new folders.
 3. Name one as **Client**.
 4. Name another one as **Server**.
 5. Now open that folder in VS Code.
- Client setup (installing react app)
 - Open the Client folder in the terminal of VSCode.
npm create vite@latest . --template react
 - Select React framework from the given options.
 - Select JavaScript variant from the given options.
 - Now lets navigate to the client folder by giving the following command.
cd client
 - To install all the packages run the following command.
npm install
 - To start the React server type the following command.
npm run dev
- Server setup (npm init)
 - Open Server folder in terminal of VScode.
npm init -y
 - Create files:

- server.js
- Create folders:
models, controllers, routes

BACKEND DEVELOPMENT:

Controllers

- adminController.js → Handles all admin-related business logic (login, manage events, clubs, etc.).
- userController.js → Handles all user-related operations (register, login, RSVP, forum posts, etc.).

db

- config.js → Contains database connection setup (e.g., MongoDB/SQL config).

middleware

- authMiddleware.js → Middleware for authentication/authorization (JWT/session validation).
- upload.js → Middleware for handling file uploads (e.g., event images, profile pics).

models

- Admin
 - Admins.js → Schema/model for admin accounts.
- User
 - Users.js → Schema/model for user accounts.
- Community
 - Events.js → Schema/model for storing campus events.
 - Clubs.js → Schema/model for storing club details.
 - Posts.js → Schema/model for user forum posts.

routes

- adminRoutes.js → Defines API routes for admin actions (login, approve clubs, delete posts, etc.).
- userRoutes.js → Defines API routes for user actions (register, login, RSVP, create post, etc.).

DATABASE DEVELOPMENT:

1. Configure MongoDB:
 - Install Mongoose. In your Node.js project directory, run: `npm install mongoose`
 - Create database connection.
 - Create Schemas & Models.

Create Database connection:

```
const mongoose = require('mongoose');
```

```

const dotenv = require('dotenv');
dotenv.config();

const connectDB = async () => {
  try {
    await mongoose.connect(process.env.MONGO_URI, {
      useNewUrlParser: true,
      useUnifiedTopology: true,
    });
  } catch (error) {
    process.exit(1);
  }
};
module.exports = connectDB;

```

Create Schemas:

1. Users.js → Schema/model for user accounts.

```

const mongoose = require('mongoose');
const UserSchema = new mongoose.Schema({
  name: String,
  email: String,
  password: String,
  major: String,
  userId: {
    type: mongoose.Schema.Types.ObjectId,
    ref: "user",
  }
})
module.exports = mongoose.model('user', UserSchema)

```

2. Events.js → Schema/model for storing events.

```

const mongoose = require('mongoose');
const eventSchema = new mongoose.Schema({
  title: String,
  description: String,
  date: String,
  location: String,
  organizerId: { type: mongoose.Schema.Types.ObjectId, ref: 'Club' },
})
module.exports = mongoose.model('events', eventSchema)

```

FRONT-END DEVELOPMENT:

Admin (src/Admin)

- Ahome.jsx → Admin dashboard/homepage.
- Alogin.jsx → Admin login page.
- Anavbar.jsx → Navigation bar for admin panel.
- ManageEvents.jsx → UI for admin to approve or delete events.
- Users.jsx → Admin panel to manage registered users.
- Clubs.jsx → Admin panel to manage campus clubs.

User (src/User)

- Uhome.jsx → User homepage/dashboard.
- Ulogin.jsx → User login page.
- Unavbar.jsx → Navigation bar for user side.
- Usignup.jsx → User signup/registration page.
- Events.jsx → Displays a directory of upcoming campus events.
- Forums.jsx → Component for students to read and post academic queries.
- MyProfile.jsx → Displays user details and their RSVP history.

Components (Reusable parts)

- Footer.jsx → Footer component used across pages.
- Home.jsx → Main home/landing page (probably entry page for app).
- ui/ → Likely contains shared UI components (buttons, modals, etc.).

Output ScreenShots:

- **User Login Page:** A clean interface requiring Email Address and Password with a secure "Log in" button.
- **Admin HomePage:** A dashboard displaying metric cards for Total Users, Active Events, and Registered Clubs, accompanied by a bar chart showing engagement overview.
- **Event Directory Page:** A grid layout listing upcoming campus activities, showing the event poster, title, date, and a "RSVP Now" button.
- **My Profile Page:** Shows the student's information and a timeline of the events they have registered to attend.

Demo Video Link:

 Campus Connect.mp4.mp4